1. Platform Architecture Flow (Correct Industrial Model)

Industrial engineering tools are not step wizards.

They behave like modular workspaces.

Your platform should operate as independent engineering modules connected by shared system data.

Core System Model

Everything revolves around the Plant Knowledge Graph.

```
Plant Graph (Neo4j)
    │
    ├── Control Loops
    ├── IO Mapping
    ├── Control Logic
    ├── Simulation
    ├── Monitoring
    ├── Root Cause Analysis
    └── Replay
```

This means:

Engineers can jump anywhere.

Example flows:

Upload Docs → Plant Graph → Control Loops → Logic

or

Plant Graph → IO Mapping → Deploy

or

Monitoring → Root Cause → Replay

Nothing is forced.

UI Navigation Model

Left Sidebar:

PROJECTS
- Plant Model
- Control Loops
- IO Mapping
- Control Logic
- Simulation
- Runtime
- Monitoring
- Diagnostics

Right Panel Tabs:

Details
Signals
Trace
Replay
IO Mapping
Control Loops
Diagnostics

Your docs already define the panel architecture.

Trigger Model (Buttons)

Each module triggers its own automation.

Example:

Upload Documents
Parse Plant Model
Detect Control Loops
Generate Logic
Generate IO Mapping
Deploy Runtime
Start Monitoring
Analyze Fault
Replay Event

Each button runs one backend service.

---

2. Control Loop Discovery Engine (FULL VERSION)

Purpose

Automatically identify industrial control loops.

Example loop:

Sensor → Process → Actuator

Example:

LT101 → Tank → Pump101

Control Loop Database Schema

PostgreSQL

control_loops
---
id
loop_tag
sensor_tag
actuator_tag
process_unit
controller_tag
loop_type
control_strategy
setpoint_tag
output_tag
status
confidence
created_at

Example record:

LOOP_101
LT101
PUMP101
TANK101
PID_LT101
LEVEL
PID
SP_LT101
OUT_PUMP101
active
0.94

Control Loop Discovery Algorithm

File

services/control_loop_engine.py

Python (~120 lines)

```
from neo4j import GraphDatabase
import uuid

class ControlLoopEngine:
```

```
def __init__(self, uri, user, password):
    self.driver = GraphDatabase.driver(uri, auth=(user,password))

def detect_loops(self):

    query = """
    MATCH (s:Sensor)-[:MEASURES]->(p:Process)
    MATCH (a:Actuator)-[:CONTROLS]->(p)
    RETURN s.tag as sensor,
           p.tag as process,
           a.tag as actuator
    """

    loops = []

    with self.driver.session() as session:
        results = session.run(query)

        for r in results:

            loop_id = "LOOP_" + str(uuid.uuid4())[:6]

            loop = {
                "loop_tag": loop_id,
                "sensor_tag": r["sensor"],
                "process_unit": r["process"],
                "actuator_tag": r["actuator"],
                "loop_type": "feedback",
                "control_strategy": "PID",
                "confidence": 0.9
            }

            loops.append(loop)

    return loops
```

Store Loops

File

services/control_loop_store.py

```
import psycopg2

def store_loops(loops):

    conn = psycopg2.connect("dbname=automation")

    cur = conn.cursor()

    for l in loops:

        cur.execute("""
        INSERT INTO control_loops
        (loop_tag,sensor_tag,actuator_tag,process_unit,
        loop_type,control_strategy,confidence)

        VALUES (%s,%s,%s,%s,%s,%s,%s)
        """,
        (
        l["loop_tag"],
        l["sensor_tag"],
        l["actuator_tag"],
        l["process_unit"],
        l["loop_type"],
        l["control_strategy"],
        l["confidence"]
        ))

    conn.commit()
```

Total code size:

~120 lines.

Control Loop Panel (Production Table)

Right Panel

CONTROL LOOPS

Table columns:

Loop ID
Sensor
Process
Actuator
Controller
Loop Type
Strategy
Setpoint
Output
Confidence
Status
Actions

Example:

LOOP101 LT101 Tank101 Pump101 PID_LT101 Level PID SP_LT101 OUT_PUMP101 0.93 Active

---

Table Actions

Buttons per row:

View Loop
Edit Strategy
Generate Logic
Trace Loop
Simulate

Frontend Table (React)

File

ControlLoopTable.jsx
<Table columns={columns} dataSource={loops}>

Columns:

Loop ID

Sensor
Process
Actuator
Strategy
Confidence
Status

Loop Graph Highlight

When clicking row:

Graph highlights:

Sensor
↓
Controller
↓
Actuator
↓
Process

Uses the same React Flow graph already used in the UI.

---

3. Root Cause Analysis Engine (FULL BUILD)

Purpose:

Automatically determine why an alarm happened.

Uses:

plant graph
signal history
control loops
logic

Data Sources

Neo4j → device relationships
InfluxDB → signal history
PostgreSQL → control loops

Root Cause Algorithm

File

services/root_cause_engine.py

Python (~140 lines)

```
from neo4j import GraphDatabase
from influxdb_client import InfluxDBClient

class RootCauseEngine:

    def __init__(self, neo4j_uri, influx_url):

        self.neo = GraphDatabase.driver(neo4j_uri)
        self.influx = InfluxDBClient(url=influx_url)

    def analyze_alarm(self, alarm_tag):

        timeline = []

        signal_history = self.get_signal_history(alarm_tag)

        cause = None

        for signal in signal_history:

            if signal["value"] > signal["threshold"]:
                cause = signal["tag"]

        device_path = self.trace_device(cause)

        return {
            "alarm": alarm_tag,
```

```
        "root_cause": cause,
        "path": device_path,
        "timeline": signal_history
    }

def trace_device(self, tag):

    query = """
    MATCH path=(n {tag:$tag})-[*]->(m)
    RETURN path
    """

    with self.neo.session() as session:
        result = session.run(query, tag=tag)

        for r in result:
            return r["path"]
```

Root Cause Output

Example:

ALARM
High Tank Level

System finds:

Valve201 stuck
Pump101 running
Tank level rising

Root cause:

Valve201 actuator failure

Root Cause UI Panel

Right Panel → Diagnostics

FAULT ANALYSIS

Display:

Alarm
Tank High Level

Root Cause
Valve201 failed to open

Confidence
92%

Affected Devices
Valve201
Pump101
Tank101

Timeline Visualization

12:01 Valve201 closed
12:02 Pump101 running
12:03 Tank level rising
12:04 High level alarm

Graph Highlight

Fault path:

Valve201 → Tank101 → LevelSensor

Highlighted in graph.

Button Trigger

Top Toolbar

Analyze Fault

Calls:

POST /analyze_fault

Backend Endpoint

FastAPI

```
@app.post("/analyze_fault")
def analyze_fault(alarm_tag):

    engine = RootCauseEngine()

    result = engine.analyze_alarm(alarm_tag)

    return result
```

Code Size

Control Loop Engine

~110 lines

Root Cause Engine

~140 lines

Both very small.

Final Architecture After These Additions

Your system becomes:

documents
↓
plant graph
↓
control loop discovery
↓
simulation
↓
logic generation
↓
IO mapping
↓
runtime
↓
monitoring
↓
root cause analysis
↓
system replay

Result

Your platform now includes:

Plant Digital Twin
Control Loop Detection
PLC Logic Generation
IO Engineering
Runtime Deployment
Monitoring
Replay Debugging
Root Cause Analysis
Version Control
Project Lifecycle

That is essentially:

TIA Portal
+ AspenTech

+ Ignition
+ OSIsoft
+ Git

inside one environment.

1 Add Enterprise Project Engineering Manager

This feature is absolutely necessary for a production industrial platform.

Right now your UI concept includes a project navigator, but it does not yet include a true project lifecycle manager.

Your interface shows plant components, but it does not manage multiple projects simultaneously across an organization.

Industrial engineering teams typically operate with:

- dozens of plants
- hundreds of PLCs
- multiple engineering teams
- multi-year system lifecycles

Because of this, the platform needs a Projects System.

Enterprise Project Manager (Developer Specification)

Purpose

The project manager manages multiple industrial automation projects within the platform.

Each project contains the complete engineering environment for a plant or system.

Each project stores:

plant graph

control logic

simulation model

IO mapping

documents

deployment states

monitoring configurations

versions

This structure allows engineers to isolate work between different plants or systems while still using the same platform.

Project Data Structure

Projects are stored in PostgreSQL.

Example table:

projects

---

id

name

description

industry

created_at

owner

status

active_version

This table allows the system to track project ownership, status, and the currently deployed automation version.

Project Folder Layout

Each project also maintains its own workspace in storage.

Example project structure:

```
/projects
  /project_001
    /documents
    /plant_graph
    /control_logic
    /simulation_models
    /io_mapping
    /runtime
    /monitoring
```

/snapshots

This directory structure mirrors the engineering lifecycle of an industrial automation system.

It also integrates directly with the platform’s automated version control system.

UI Layout

The left sidebar becomes a project-aware navigation system.

Example structure:

PROJECTS

```
├ Water Treatment Plant
│   ├ Documents
│   ├ Plant Model
│   ├ Control Logic
│   ├ IO Mapping
│   ├ Simulation
│   ├ Monitoring
│
├ Packaging Line
│   ├ Documents
│   ├ Plant Model
│   ├ Control Logic
│   ├ IO Mapping
│   ├ Simulation
```

│ ├ Monitoring

Each project contains its own engineering workspace.

Switching projects switches the entire plant context.

Project Creation Workflow

Engineers create projects directly inside the platform.

User action:

Create Project

This opens a configuration form.

Form fields:

Project Name

Industry Type

Plant Description

Import Documents

Select PLC Runtime

Once the project is created, the engineering pipeline begins automatically once the engineer triggers individual modules.

Open Source Stack

The project management layer uses the same stack as the rest of the platform.

Core technologies:

FastAPI

PostgreSQL

Docker volumes

Git snapshot system

This keeps the platform fully self-hosted and enterprise deployable.

Enterprise Features

The project system should also include enterprise capabilities such as:

project permissions

project version branching

multi-user editing

audit logs

These features allow multiple engineers to collaborate safely while maintaining traceability across the system lifecycle.

Vendor-Agnostic Code Export Engine

(Developer Specification)

Purpose

Allow engineers to export generated control logic from the platform into multiple industrial PLC vendor formats.

The platform internally maintains vendor-agnostic Structured Text (ST).

When engineers need to deploy to real hardware, the system converts the vendor-agnostic logic into the appropriate vendor format.

Supported targets:

Rockwell (Studio 5000)
Siemens (TIA Portal)
Codesys
Beckhoff TwinCAT
OpenPLC

The export system must remain:

- fully open source
- modular
- extensible

Core Architecture

Your software keeps one canonical logic format:

Vendor-Agnostic Structured Text

Example internal logic:

```
PROGRAM Pump_101

IF LT101 > 90 THEN
   PUMP101 := FALSE;
END_IF;

END_PROGRAM
```

This logic is stored in your platform's project structure.

/control_logic

```
main.st
equipment/
sequences/
control_loops/
interlocks/
alarms/
```

This architecture already matches the dynamic logic generation system in your platform.

Export Pipeline

When engineers click Export, the platform performs the following pipeline:

Vendor-Agnostic ST
↓
Logic Parser
↓
Intermediate Logic Model
↓
Vendor Template Engine
↓
Vendor Project Generator
↓
Downloadable Project

This ensures your platform never becomes locked to one PLC ecosystem.

Export Engine Workflow

Engineer action in UI:

Export Logic

Export dialog opens:

Export Target
———————————

Rockwell Studio 5000

Siemens TIA Portal
Codesys
TwinCAT
OpenPLC

Engineer selects vendor.

System runs export pipeline automatically.

Output:

Download Vendor Project

Core Open Source Tools

The export engine relies entirely on open source components.

Structured Text Parsing

Tool:

Tree-Sitter Structured Text Grammar

Purpose:

- parse vendor-agnostic ST
- create syntax tree
- allow transformation

Logic Transformation Layer

Language:

Python

Library examples:

lark-parser
tree-sitter

Used for:

parse ST
build intermediate model
generate vendor templates

Template Engine

Tool:

Jinja2

Purpose:

Generate vendor-specific code templates.

Example:

Rockwell AOI template
Siemens FB template
Codesys POU template

Vendor Export Modules

Each vendor uses its own export module.

Structure:

```
/export_engine
    /rockwell
    /siemens
    /codesys
    /beckhoff
    /openplc
```

Each module includes:

logic_templates

project_templates
tag_templates

Example: Siemens Export

Generated files:

```
/TIA_Project
   /ProgramBlocks
      Pump_101.scl
      Valve_201.scl
   /Tags
      plant_tags.xml
```

The exporter packages these files into:

TIA_Project.zip

Engineer imports into TIA Portal.

Example: Rockwell Export

Generated files:

```
/Studio5000_Project
   /AOIs
   /Programs
   /Tags
```

Output:

Studio5000_L5X.xml

This format is the official Rockwell import format.

Export Engine Code Skeleton

File:

services/export_engine.py

Python (~120 lines)

```
import os
from jinja2 import Environment, FileSystemLoader

class ExportEngine:

    def __init__(self):

        self.template_env = Environment(
            loader=FileSystemLoader("export_templates")
        )

    def export(self, vendor, logic_files):

        if vendor == "siemens":
            return self.export_siemens(logic_files)

        if vendor == "rockwell":
            return self.export_rockwell(logic_files)

    def export_siemens(self, logic_files):

        template = self.template_env.get_template("siemens_block.j2")

        output_files = []

        for file in logic_files:

            rendered = template.render(code=file)

            output_files.append(rendered)

        return output_files

    def export_rockwell(self, logic_files):

        template = self.template_env.get_template("rockwell_block.j2")
```

```
    output_files = []

    for file in logic_files:

        rendered = template.render(code=file)

        output_files.append(rendered)

    return output_files
```

UI Integration

Top Toolbar

Add button:

Export Logic

Clicking opens export panel.

Export Panel:

EXPORT PROJECT

Target Vendor
_____

Siemens TIA Portal
Rockwell Studio 5000
Codesys
TwinCAT
OpenPLC

[Generate Export]

Export Output

After generation:

Download Project Package

File examples:

TIA_Project.zip
Studio5000_L5X.xml
Codesys_Project.project

Integration With Existing Platform

The export engine integrates with:

control loop engine
logic generation engine
IO mapping
version control
project system

Example pipeline:

Plant Graph
↓
Control Loops
↓
Logic Generation
↓
IO Mapping
↓
Export Engine
↓
Vendor Project

Why This Architecture Is Powerful

Your platform becomes the vendor-agnostic automation IDE.

Vendors become export targets, not the primary environment.

This is similar to how:

LLVM compiles to many CPUs

Your platform compiles to:

Siemens
Rockwell
Codesys
Beckhoff
OpenPLC

Result

Your platform now contains:

Plant Model
Control Loop Engine
Logic Generator
IO Mapping
Simulation
Runtime Deployment
Monitoring
Replay
Root Cause Analysis
Version Control
Project Manager
Export Engine

Which effectively makes your software:

TIA Portal
+ Studio 5000
+ Ignition
+ AspenTech
+ Git
+ Digital Twin

inside one environment.

Automatic PLC Project Reverse Engineering

Purpose

Allow engineers to upload an existing PLC project and automatically reconstruct:

- plant structure
- control loops
- IO mappings
- signal relationships
- logic structure

The system converts the project into the platform’s internal Plant Graph + Logic Model.

This enables:

• migration from legacy PLC environments

• analysis of existing plants

• digital twin generation from deployed systems

• reverse engineering undocumented control systems

The resulting output integrates directly with the platform's engineering pipeline.

Target Input Sources

The system must support common PLC project formats:

Rockwell Studio 5000 (.L5X / .ACD)
Siemens TIA Portal (.XML / .AP16)
Codesys (.project / .xml)
Beckhoff TwinCAT (.tsproj)
OpenPLC (.st)
Generic IEC Structured Text
Generic Ladder Logic
CSV IO tables

The engine must also support unstructured projects such as:

loose ST files
partial logic exports
CSV tag lists
unknown vendor formats

Reverse Engineering Pipeline

PLC Project Upload
↓
Project Format Detection
↓
Logic Extraction
↓
Signal Extraction
↓
IO Mapping Reconstruction
↓
Control Loop Detection
↓
Plant Graph Reconstruction
↓

Logic Normalization
↓
System Model Generation

Output becomes:

Plant Graph
Control Logic
Control Loops
IO Mapping
Signals

Which immediately integrates with the platform.

Core Open Source Tools

Structured Text Parsing

Tree-Sitter IEC 61131-3 Grammar

Purpose:

• parse Structured Text

• detect logic structure

• extract variables

Ladder Logic Parsing

Lark Parser
Custom IEC Grammar

Used to parse ladder logic XML exports.

Graph Reconstruction

Neo4j

Rebuild relationships between:

sensors
actuators
logic blocks
control loops

Data Processing

Python
Pandas

Used for:

• tag extraction

• signal classification

• device grouping

Handling Sloppy or Unstructured Projects

Real industrial PLC projects are often:

• poorly named

• inconsistent

• missing documentation

• partially structured

To ensure robustness, the system includes several normalization layers.

Robustness Layer 1

Tag Normalization Engine

Purpose:

Standardize inconsistent tag names.

Example:

LT101
LT-101
LEVEL_101
TANK_LEVEL_SENSOR

Normalized to:

LT101

Implementation:

regex normalization
pattern classification
dictionary mapping

Tools:

Python
Pandas
Regex

Robustness Layer 2

Signal Classification Engine

Automatically determine signal type.

Example:

LT101 → Level Sensor
PUMP101 → Motor
VALVE201 → Actuator

Uses rule engine:

Durable Rules

Classification example:

if tag starts with LT → Level Sensor
if tag contains VALVE → Valve
if tag contains PUMP → Pump

Robustness Layer 3

Graph Inference Engine

Detect hidden system relationships.

Example logic:

IF LT101 > 90 THEN PUMP101 := FALSE

Inference:

LT101 → controls → PUMP101

Graph edge generated automatically.

Robustness Layer 4

Control Loop Discovery

Automatically detect feedback loops.

Example:

Sensor → Controller → Actuator

Detected loops:

LT101 → PID → PUMP101

These loops populate the Control Loop Table.

Robustness Layer 5

Fuzzy Device Matching

Used for messy legacy projects.

Example:

pump_a
pmp101
pumpMain

Mapped to:

Pump101

Tools:

RapidFuzz

Reverse Engine Output

After reconstruction the platform generates:

Plant Graph
Control Loops
IO Mapping
Logic Files
Device Metadata

These are stored in the project workspace.

UI Integration

New import option:

Import Existing PLC Project

Supported uploads:

PLC Project File
Logic Export
Tag List
IO Table

Processing runs automatically.

The result appears in:

Plant Model
Control Loops
IO Mapping
Logic Editor

Performance Design

The system must remain fast.

Techniques used:

parallel parsing
incremental graph building
lazy device classification
cached tag dictionaries

Typical import speed:

1,000 tags → < 5 seconds
10,000 tags → < 20 seconds

Future Capability

Direct PLC Deployment Layer

Purpose

Allow the platform to deploy logic directly to PLC hardware without requiring vendor IDEs such as:

TIA Portal
Studio 5000
TwinCAT
Codesys IDE

The platform becomes the primary engineering environment.

Deployment Architecture

Vendor-Agnostic Logic
↓
Export Engine
↓
Protocol Driver
↓
PLC Runtime

The system communicates with PLCs using open industrial protocols.

Open Protocols

The deployment engine supports:

Modbus TCP
OPC UA
EtherNet/IP
Profinet
MQTT Industrial

These protocols allow communication with most industrial PLCs.

Core Open Source Tools

OPC UA

open62541
FreeOpcUa

Used for:

Siemens
Beckhoff
Codesys
modern PLCs

Modbus

pymodbus
libmodbus

Used for:

industrial PLCs
embedded controllers

EtherNet/IP

pycomm3
cpppo

Used for:

Rockwell PLCs
Allen Bradley devices

Profinet

p-net stack

Used for:

Siemens industrial networks

Deployment Engine Workflow

Engineer clicks:

Deploy to PLC

Pipeline executes:

Logic Compilation
↓
IO Mapping
↓
Protocol Mapping
↓
Device Connection
↓
Runtime Upload

Deployment Output

The system sends:

logic blocks
tag tables
IO mapping
runtime configuration

to the PLC.

Runtime Support

Supported runtime targets:

OpenPLC
Codesys Runtime

Industrial Edge Controllers
Embedded Linux PLCs

This allows the platform to run the automation system without vendor IDEs.

Deployment UI

Top Toolbar:

Deploy PLC

Dialog:

PLC Address
Protocol
Target Runtime
IO Configuration

Example:

IP: 192.168.1.10
Protocol: OPC UA
Runtime: OpenPLC

Safety Layer

Before deployment the system runs:

syntax validation
logic verification
IO validation
simulation test

Deployment only occurs if validation passes.

Resulting Platform Capabilities

After these future modules the platform becomes capable of:

Import Existing PLC Projects
Reverse Engineer Plant Logic
Generate Digital Twin
Generate Control Logic
Simulate System
Deploy Directly to PLC
Monitor Plant
Analyze Failures
Replay System Behavior

Which effectively creates a complete vendor-agnostic automation platform.

Step 4.2 — P&ID Reconciliation Engine

(Iterative Engineering Update Layer)

Purpose

Industrial automation projects rarely operate from a single finalized P&ID.

Instead, the engineering workflow typically involves multiple iterative revisions between process engineers and automation engineers.

Example lifecycle:

Initial P&ID draft
↓
Instrumentation review
↓
Automation feedback
↓
Updated P&ID revision
↓
Control strategy updates
↓
Instrumentation additions or changes

If the automation pipeline interprets only a static P&ID snapshot, the system cannot correctly track engineering changes.

The P&ID Reconciliation Engine solves this problem by automatically detecting and integrating changes between document revisions.

The engine performs the following tasks:

• detect new instrumentation

• detect removed devices

• reconcile messy tag naming

• update plant topology

• maintain historical plant model versions

• prevent destructive changes to existing control logic

This allows the platform to support continuous engineering iteration, which reflects real industrial project workflows.

Where This Integrates in the Pipeline

This feature is inserted after Step 4 — Data Normalization and before Step 4.5 — Engineering Validation.

Updated workflow:

documents
↓
data extraction
↓
plant graph
↓
signal trace
↓
data normalization
↓
P&ID reconciliation
↓
engineering validation
↓
simulation
↓
control logic generation

This ensures the dataset is reconciled before validation and simulation.

Tools (All Open Source)

The reconciliation layer intentionally reuses existing infrastructure to keep implementation extremely small.

Core tools:

Data Processing

Python

Pandas

Used for dataset comparison and tag normalization.

Graph Update Engine

Neo4j

Python Neo4j Driver

Updates plant graph nodes and relationships.

Data Validation

JSON Schema Validator

Ensures updated dataset remains valid.

Version Tracking

Git

GitPython

Dolt

Automatically records plant model updates.

Backend API

FastAPI

Handles document revision processing requests.

Workflow

Step 1 — Load Current Plant Model

The engine loads the current plant model from Neo4j.

Example query:

```
MATCH (n)
RETURN n.tag, n.type, n.relationships
```

The current plant configuration is stored as a dataset.

Step 2 — Parse New Document Dataset

The latest document upload has already passed through:

- OCR extraction
- metadata extraction
- normalization

The dataset contains structured instrumentation metadata.

Example:

tag: LT101
type: level_transmitter
equipment: Tank101

Step 3 — Tag Normalization

Messy tag formats are normalized automatically.

Examples handled:

LT101
LT-101
LIT101
Level101

Normalization rules:

• convert to uppercase

• enforce hyphen formatting

• remove whitespace

• infer instrument type prefix

Example rule:

LT101 → LT-101

Step 4 — Change Detection

The engine compares the new dataset with the existing plant graph.

Three change types are detected.

New Devices

Example:

FT-203 detected in new P&ID

Action:

create Neo4j node
attach device metadata
connect to equipment

Removed Devices

If an existing device disappears in the updated dataset:

existing node: LT-105
not present in new dataset

Instead of deleting the node:

status = deprecated

This prevents breaking previously generated logic.

Topology Changes

Example:

Old topology

Pump101 → Tank101

New P&ID revision

Pump101 → Valve201 → Tank101

The system updates the Neo4j graph edges accordingly.

Step 5 — Duplicate Tag Resolution

OCR and engineering documents often produce duplicate tags.

Example:

LT101
LT-101
LT101A

The system calculates similarity using string distance.

If similarity exceeds threshold:

possible duplicate detected

Conflict is flagged for engineer review.

Step 6 — Validation Layer

After reconciliation, the updated dataset passes through the existing validation layer.

Validation checks include:

• duplicate tags

• orphan sensors

• broken equipment connections

• invalid signal types

This prevents corrupted plant graphs.

Step 7 — Graph Update

The final dataset is written to Neo4j.

Example operations:

```
MERGE (n:Sensor {tag:"LT-101"})
SET n.type="level_transmitter"
```

Graph edges are updated automatically.

Step 8 — Version Snapshot

The reconciliation engine triggers an automatic version commit.

The updated plant model is exported and committed.

Example commit message:

```
P&ID Revision Detected

+ Added FT-203
+ Added Valve-301
~ Updated Pump101 discharge path
- Deprecated LT-105
```

Version tracking is handled by the existing Git and Dolt version system.

UI Integration

The feature integrates into the existing Automation Build Studio UI layout.

The UI uses a three-panel architecture:

Top Toolbar
Left Project Navigator
Main Workspace
Right Engineering Panel

New UI Panel

Add a new tab to the Right Engineering Panel.

Current tabs:

Details
Signals
Trace
Replay
Diagnostics

Updated tabs:

Details
Signals
Trace
Replay
P&ID Changes
Diagnostics

P&ID Changes Panel

Example UI view:

```
P&ID REVISION ANALYSIS
--------------------------------

New Devices
+ FT-203
+ Valve-301

Modified Connections
Pump101 → Valve201 → Tank101

Deprecated Devices
LT-105

Possible Tag Conflicts
LT101 vs LT-101
```

Engineer actions:

Accept Changes
Review Conflicts
Apply Update

Create Version Snapshot

Settings

Add new configuration group.

Settings → P&ID Processing

Options:

Enable P&ID Reconciliation: ON
Tag Similarity Threshold: 0.85
Auto Normalize Tags: ON
Allow Automatic Node Creation: ON
Mark Missing Devices Deprecated: ON
Require Engineer Approval: ON

Development Effort

Estimated implementation size:

Python reconciliation logic: ~100 lines
API endpoint: ~25 lines
UI panel: ~50 lines

Total:

~150–180 lines

This remains extremely small because it reuses:

- Neo4j graph
- existing normalization layer
- validation layer

- versioning system

Result

The automation platform now supports iterative engineering development workflows, not just static P&ID interpretation.

Capabilities added:

• automatic reconciliation of P&ID revisions

• support for messy engineering documents

• safe plant model updates

• automatic version tracking of engineering changes

The platform can now evolve alongside engineering design, which is essential for real industrial automation projects.